PROCESSAMENTO DE ALTO DESEMPENHO

Atividade Remota Avançada
Otimização Sequencial e Desempenho de Memória

Nível: alto | Modalidade: individual | Linguagem: Python

1. Objetivo geral

Nesta atividade, o aluno deverá investigar o desempenho de diferentes implementações de um mesmo problema, identificar gargalos computacionais e justificar tecnicamente as otimizações realizadas. O objetivo não é apenas produzir um código mais rápido, mas responder:

"Por que determinada implementação é mais eficiente e qual componente do sistema limita seu desempenho?"

- analisar complexidade computacional e medir desempenho de forma adequada;
- utilizar profiling para identificar gargalos;
- distinguir custo computacional de custo de acesso à memória;
- investigar localidade espacial, temporal e layouts de memória;
- aplicar técnicas de loop optimization, vetorização e JIT;
- discutir trade-offs entre tempo, memória e precisão numérica.

2. Problema principal

Considere uma aplicação científica que trabalha com uma matriz bidimensional representando um campo físico:

$$A \in \mathbb{R}^{N \times N}$$

Cada elemento interno deve ser atualizado utilizando seus quatro vizinhos e o próprio valor:

$$B_{ij} = (A_{i-1,j} + A_{i+1,j} + A_{i,j-1} + A_{i,j+1} + A_{ij}) / 5$$

$$1 \leq i < N - 1 \quad e \quad 1 \leq j < N - 1$$

Esse tipo de operação (stencil 2D) aparece em diferenças finitas, difusão, suavização espacial, processamento de imagens, solução iterativa de equações diferenciais e modelos ambientais.

Parte I — Implementação de referência

Questão 1 — Implementação e benchmark

```
import numpy as np

def stencil_v1(A):
    N = A.shape[0]
```

```
    B = np.zeros_like(A)

    for i in range(1, N - 1):
        for j in range(1, N - 1):
            B[i, j] = (
                A[i - 1, j] + A[i + 1, j]
                + A[i, j - 1] + A[i, j + 1]
                + A[i, j]
            ) / 5.0

    return B
```

Execute para N = 500, 1000, 2000 e 4000 (ou ajuste os maiores tamanhos se necessário). Para cada tamanho, faça pelo menos cinco execuções e calcule:

- tempo mínimo;
- tempo médio;
- mediana;
- desvio-padrão.

Parte II — Profiling

Questão 2 — Identificação de gargalos

Utilize cProfile para investigar o código.

```
python -m cProfile programa.py
```

ou:

```
import cProfile
```

- identifique as funções que consomem mais tempo;
- verifique quantas chamadas são realizadas;
- discuta se o maior custo está em funções específicas ou nos próprios laços.

Responda: (a) qual é o principal gargalo? (b) o profiling confirma sua expectativa inicial? (c) por que profiling é superior a observar apenas o tempo total?

Parte III — Complexidade computacional

Questão 3 — Escalabilidade teórica

Determine a complexidade temporal do algoritmo. Quantos elementos internos são processados? Qual é a ordem assintótica? Discuta o efeito esperado ao dobrar N e compare com os resultados experimentais.

$$T(N) \propto N^2$$

Parte IV — Intensidade computacional

Questão 4 — Operações versus bytes

Para cada elemento calculado, estime aproximadamente o número de somas, divisões/multiplicações, valores lidos de A e valores escritos em B. Assumindo float64:

$$1 \text{ elemento} = 8 \text{ bytes}$$

Estime a quantidade de dados movimentada por atualização e discuta se o algoritmo tende a ser compute-bound ou memory-bound. Justifique.

Parte V — Eliminação de operações

Questão 5 — Divisão versus multiplicação

```
# versão original
(...) / 5.0

# alternativa
(...) * 0.2
```

Compare os tempos e discuta: existe ganho mensurável? O ganho é significativo? Por que micro-otimizações aritméticas podem ter impacto pequeno em um algoritmo dominado por memória?

Parte VI — Vetorização

Questão 6 — NumPy

```
def stencil_numpy(A):
    B = np.zeros_like(A)
    B[1:-1, 1:-1] = (
        A[:-2, 1:-1]
        + A[2:, 1:-1]
        + A[1:-1, :-2]
        + A[1:-1, 2:]
        + A[1:-1, 1:-1]
    ) * 0.2
    return B
```

Compare com a versão original e calcule o speedup para todos os tamanhos:

$$S = T_Python / T_NumPy$$

Construa um gráfico Speedup × N e explique por que NumPy pode apresentar grande ganho mesmo executando aproximadamente o mesmo algoritmo.

Parte VII — Uso de memória

Questão 7 — Estimativa de memória

$$M = 8N^2 \text{ bytes}$$

Calcule o consumo aproximado para N = 1000, 2000, 4000, 8000 e 16000. Considere pelo menos duas matrizes, A e B. Discuta em quais níveis de cache esses dados poderiam caber e quando a RAM passa necessariamente a dominar o armazenamento.

Parte VIII — Layout de memória

Questão 8 — C-order e F-order

```
A_C = np.ones((N, N), order="C")
A_F = np.ones((N, N), order="F")
```

Investigue: (1) C-order por linhas; (2) C-order por colunas; (3) F-order por linhas; (4) F-order por colunas. Explique os resultados usando memória contígua, cache line, localidade espacial e stride.

Parte IX — Stride

Questão 9 — Custo por acesso

```
A = np.ones(50_000_000, dtype=np.float64)
# testar A[::stride] para stride = 1, 2, 4, ..., 1024
```

$$t_{acesso} = T / N_{acessos}$$

Construa um gráfico t_acesso × stride. Explique por que o tempo total pode diminuir enquanto o custo por acesso aumenta, quando a localidade espacial começa a ser desperdiçada e como o tamanho da linha de cache influencia o comportamento.

Parte X — Blocking / Tiling

Questão 10 — Multiplicação matricial em blocos

$$C = A\,B$$

```
for ii in range(0, N, BS):
  for jj in range(0, N, BS):
    for kk in range(0, N, BS):
      for i in range(ii, min(ii + BS, N)):
        for j in range(jj, min(jj + BS, N)):
          for k in range(kk, min(kk + BS, N)):
            C[i, j] += A[i, k] * B[k, j]
```

Teste BS = 8, 16, 32, 64 e 128. Responda: qual bloco foi mais eficiente? Por que o tamanho ótimo depende do processador? Qual a relação com cache? Um bloco maior é sempre melhor?

Parte XI — JIT compilation

Questão 11 — Numba

```
from numba import njit

@njit
def stencil_numba(A):
  ...
```

Compare Python puro, Numba e NumPy. Desconsidere a primeira execução do Numba na medição principal, pois ela inclui a compilação JIT. Discuta por que Numba acelera os laços, por que o efeito da cache fica mais evidente após remover o overhead do interpretador e se NumPy será sempre mais rápido que Numba.

Parte XII — Otimização orientada por evidências

Questão 12 — Três otimizações sucessivas

Escolha a implementação mais lenta e faça três otimizações sucessivas. Para cada modificação: descreva a alteração, registre uma hipótese, meça o desempenho, calcule o speedup e conclua se funcionou.

Versão	Alteração	Tempo	Speedup
V0	Original		1,0
V1	Otimização 1		
V2	Otimização 2		
V3	Otimização 3		

Parte XIII — Uma otimização que falha

Questão 13 — Hipótese rejeitada

Implemente ao menos uma modificação que pareça promissora, mas que resulte em pouco ganho ou pior desempenho. Explique o resultado. Uma hipótese plausível não substitui uma medição.

Parte XIV — Escalabilidade sequencial

Questão 14 — Normalização do tempo

Para a melhor versão, execute diferentes tamanhos e construa:

$$T(N) \times N \quad e \quad [T(N) / N^2] \times N$$

Se o algoritmo for perfeitamente $O(N^2)$, $T(N)/N^2$ deveria permanecer aproximadamente constante. Discuta desvios considerando cache, RAM, alocação de memória, overhead e sistema operacional.

Parte XV — Precisão numérica

Questão 15 — Associatividade em ponto flutuante

$$(a + b) + c \quad \neq \quad a + (b + c) \quad \text{em geral, em ponto flutuante}$$

Produza um exemplo em Python em que os resultados sejam diferentes. Explique arredondamento, precisão finita e associatividade. Discuta se uma implementação mais rápida é necessariamente melhor quando altera o resultado numérico.

Parte XVI — Desafio principal

Questão 16 — Competição de desempenho

Desenvolva a implementação sequencial mais eficiente possível para o stencil. São permitidos Python, NumPy, Numba, reorganização dos laços, blocking, pré-alocação e outras otimizações sequenciais. Não é permitido multiprocessing, MPI, OpenMP, GPU ou CUDA.

$$S_{max} = T_{original} / T_{melhor\ versão}$$

Utilize N = 4000 ou o maior valor que sua máquina conseguir processar adequadamente.

Parte XVII — Análise crítica

Questão 17 — Discussão final

- Qual otimização produziu o maior ganho?
- Qual produziu o menor ganho?
- Qual resultado mais contrariou sua expectativa?
- Em quais experimentos a CPU provavelmente ficou esperando dados da memória?
- Em quais experimentos o custo do interpretador Python dominou?
- Qual a diferença entre otimizar o algoritmo e otimizar a implementação?
- Antes de paralelizar esse programa, quais otimizações sequenciais deveriam ser feitas?

Parte XVIII — Questão de síntese

Questão 18 — Paralelizar um código lento resolve o problema?

Analise criticamente a afirmação: “Paralelizar um código lento é uma forma eficiente de resolver problemas de desempenho.” Produza uma resposta técnica de aproximadamente 20 linhas, discutindo algoritmo, implementação, cache, memória, vetorização, profiling, overhead paralelo, escalabilidade e otimização sequencial.

Entregáveis

- relatório em PDF;
- todos os arquivos .py;
- resultados em tabelas;
- gráficos;
- descrição do hardware;
- profiling;
- análise de complexidade;
- discussão crítica.

Gráficos mínimos obrigatórios

- tempo × tamanho do problema;
- speedup × tamanho do problema;
- tempo por acesso × stride;
- comparação C-order × F-order;
- comparação Python × NumPy × Numba.

Informações do hardware

- processador;

- número de núcleos e threads;
- RAM;
- cache L1, L2 e L3;
- versão do Python;
- versão do NumPy;
- versão do Numba, quando utilizado.

Critérios de avaliação

Item	Peso
Implementação correta	15%
Metodologia experimental	15%
Profiling e identificação de gargalos	15%
Análise de memória/cache	20%
Otimizações realizadas	15%
Interpretação dos resultados	15%
Organização e qualidade do relatório	5%

Critério central

Resultados numéricos isolados não são suficientes. Para cada experimento deverá existir a sequência:

$$\text{Hipótese} \rightarrow \text{Experimento} \rightarrow \text{Medição} \rightarrow \text{Interpretação}$$

Uma otimização somente será considerada válida quando houver evidência experimental de sua eficácia.

Questão bônus — nível muito avançado

Intensidade aritmética e interpretação qualitativa do Roofline Model

$$I = \text{operações em ponto flutuante / bytes movimentados} \quad [\text{FLOP/byte}]$$

Calcule aproximadamente a intensidade aritmética do stencil. Pesquise as especificações aproximadas do processador utilizado e discuta qualitativamente se a aplicação tende a ser limitada por capacidade computacional ou largura de banda de memória. Explique como essa análise ajuda a decidir se investir em mais otimizações aritméticas ainda faz sentido.